Computer Sciences faculties USTHB.
2026 - 2027

Embedded Software Development

Chapter 1 : Introduction to Embedded Systems

COURSE 2 : ARM Architecture and Embedded Processors

Teacher : Dr. ABADA Lyes.
E-mail : abada.lyes@gmail.com

What will we learn?

In this course

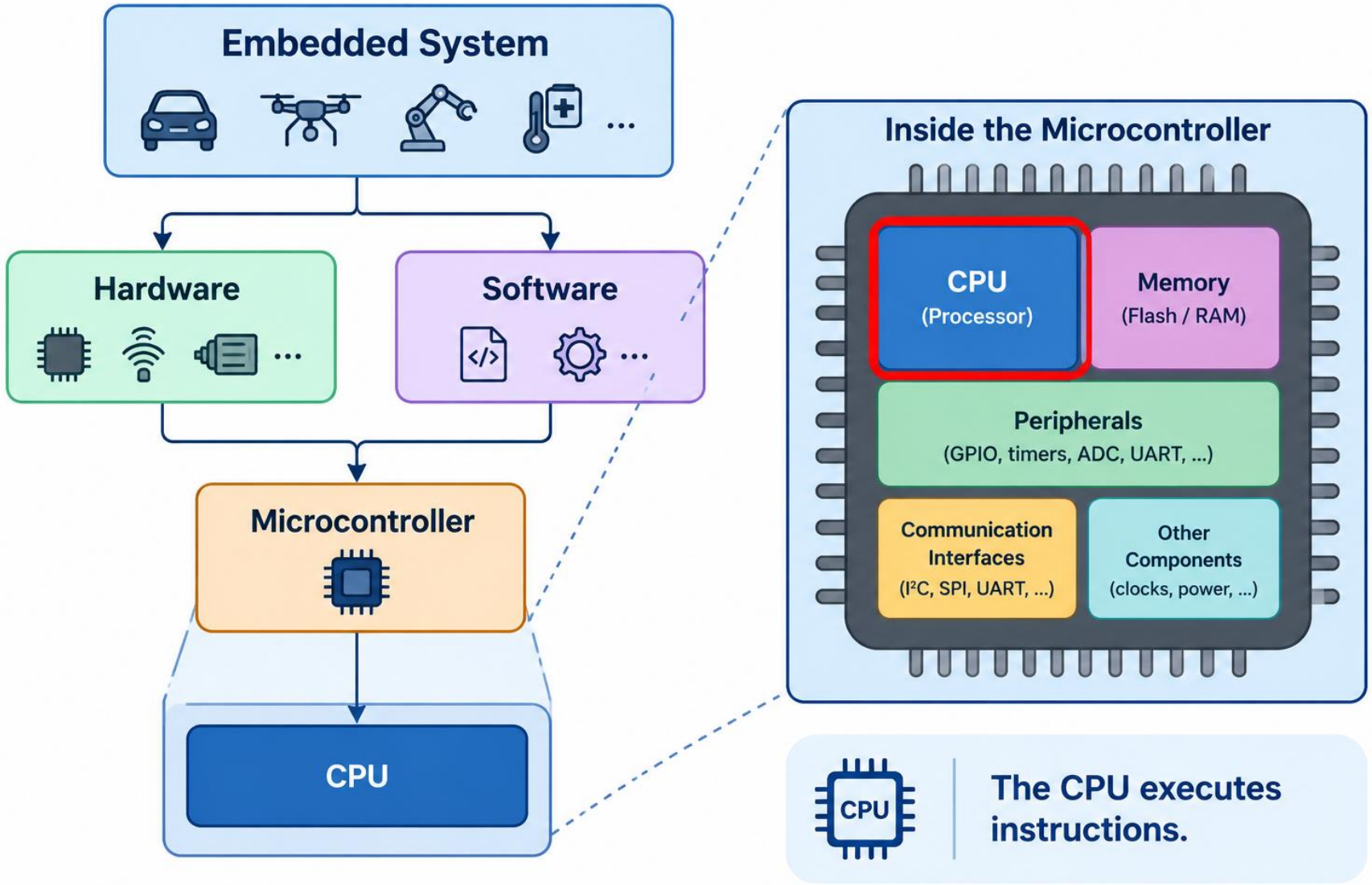
We will learn. At the end of this course, students should be able to:

- Explain the role of a processor in an embedded system.
- Identify the main ARM processor families.
- Understand the basic Cortex-M architecture.
- Identify CPU registers and their roles.
- Explain the Program Counter and Stack Pointer.
- Understand the status register.
- Explain the basic instruction execution cycle.
- Understand pipelining.
- Explain interrupts and exceptions.
- Explain the interrupt vector table.
- Understand why interrupts are important in embedded systems.

1. FROM EMBEDDED SYSTEM TO PROCESSOR

Where Is the Processor?

From the previous course:



The CPU (Central Processing Unit) executes instructions.

by : Dr. L.ABADA

1. FROM EMBEDDED SYSTEM TO PROCESSOR

What Does a CPU Do?

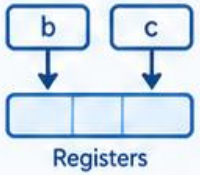
At a very simplified level:

1. **Fetch**
   Read the next instruction from memory.
2. **Decode**
   Figure out what the instruction means.
3. **Execute**
   Perform the operation (e.g., addition, load, store, ...).
4. **Store result**
   Write the result to a register or memory.
5. **Fetch next instruction**
   Go back to step 1 for the next instruction.

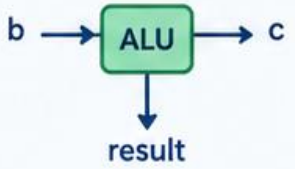
Example: `a = b + c;`

The processor must:

1. **Read the required values.**
   Load `b` and `c` from memory (or registers) into the CPU.



2. **Perform an addition.**
   Use the ALU to compute `b + c`.

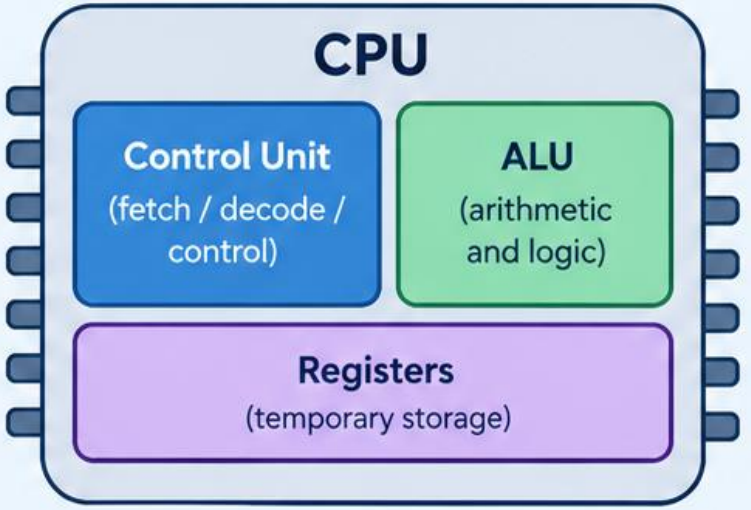


3. **Store the result.**
   Write the result into the variable `a` (in memory or a register).



The CPU repeats these steps for each instruction in the program.

Inside the CPU

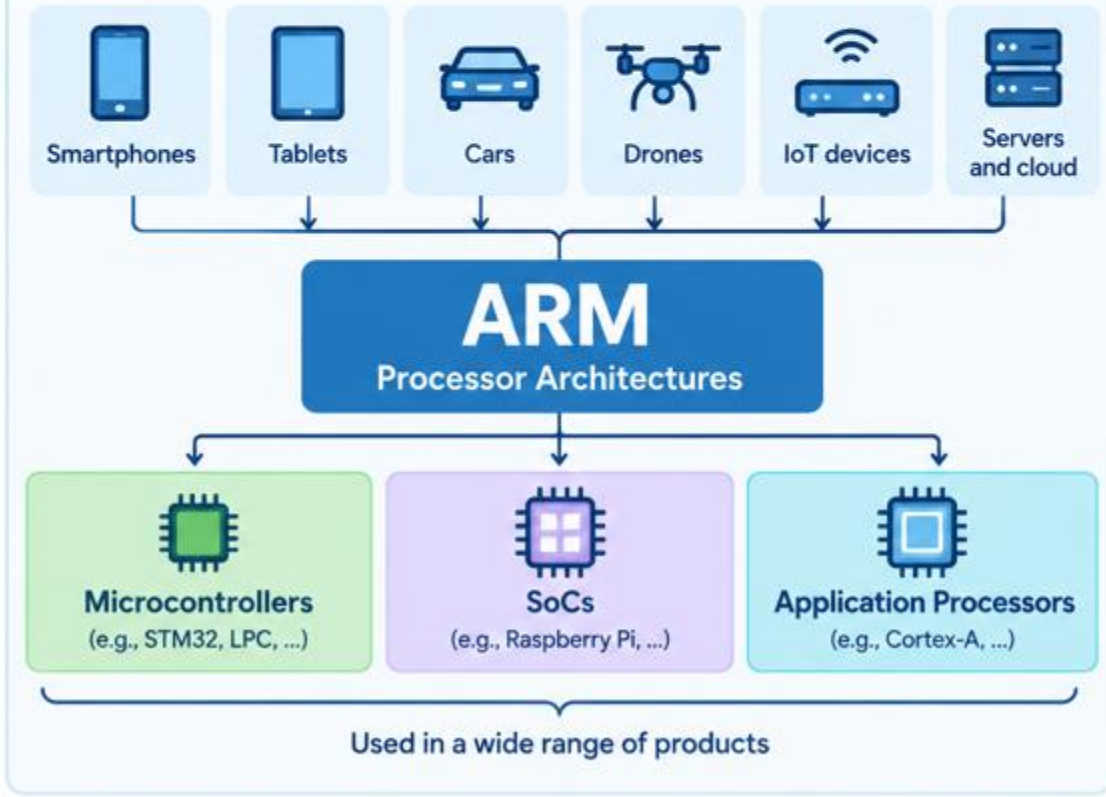


by : Dr. L.ABADA

What Is ARM?

ARM is a family of processor architectures widely used in embedded systems.

Where ARM is used



Why ARM?

Efficient instruction sets
Simple and optimized instructions for embedded applications.

Low power consumption
Designed for energy efficiency, ideal for battery-powered devices.

High performance per watt
Delivers good performance while using less power.

Large ecosystem
Supported by many tools, compilers, RTOS and software libraries.

Extensive use in microcontrollers and SoCs
From small MCUs to powerful application processors.

by : Dr. L.ABADA

ARM Is Not a Microcontroller

Important distinction:

ARM ≠ STM32

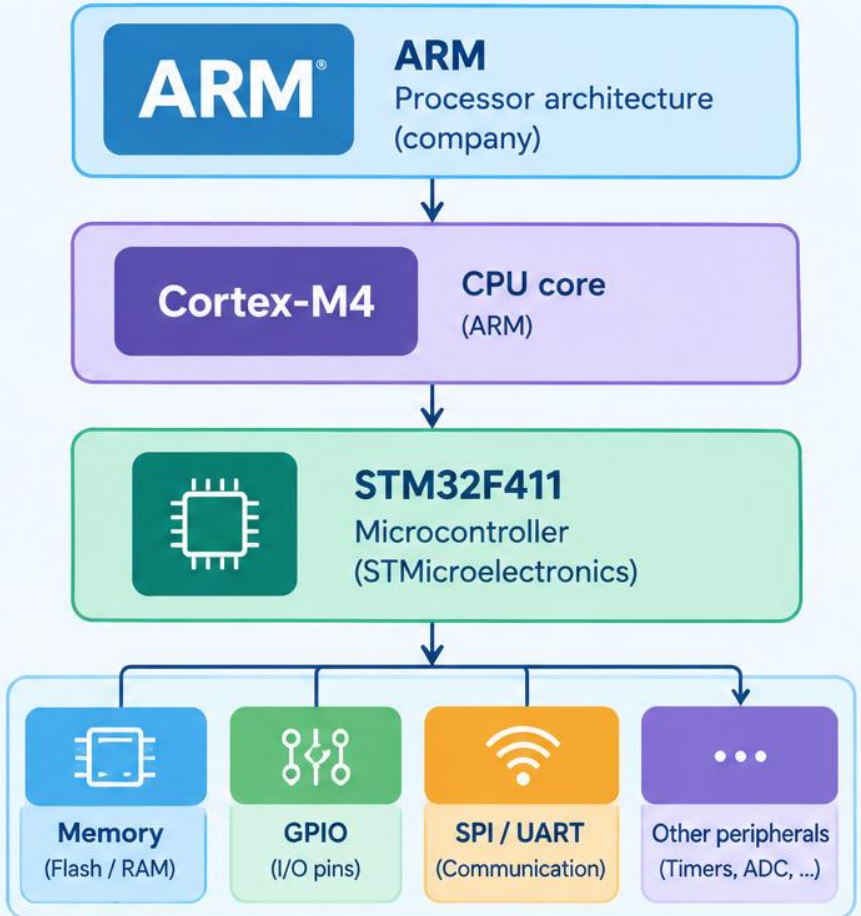
ARM
provides processor architectures and CPU cores.

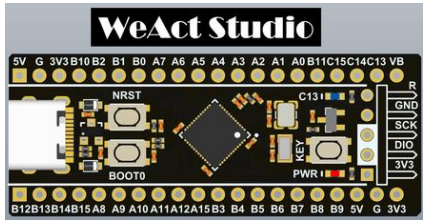
STM32
Companies such as STMicroelectronics integrate ARM Cortex cores into their microcontrollers.

In short:

ARM is the processor architecture.
STM32 is a microcontroller that uses ARM cores.

Example:



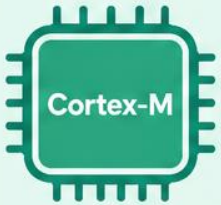
arm	**Arm — designs the Cortex-M4 core** Arm develops the processor design: its instruction execution logic, registers, pipeline, and other internal circuits. It licenses this design to semiconductor companies. **United Kingdom**
STMicroelectronics	**STMicroelectronics — designs the STM32F411 chip** ST integrates the Cortex-M4 core with Flash, SRAM, buses, and peripherals to create the complete microcontroller design. **Franco-Italian**
	**Semiconductor fabrication plant — manufactures the silicon** Specialized factories use photolithography and many other processes to build the microscopic transistors and circuits on silicon wafers. **Varies : TSMC (Taiwan), ASML(Netherlands)…**
	**WeAct Studio : development board (ex : Black Pill)** Produces the Black Pill development board, integrating the STM32F411 with the PCB and other components. **China**

Main Cortex Families

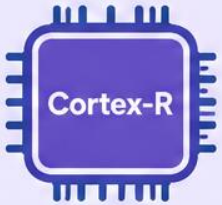
Main Cortex Families

A simplified classification:

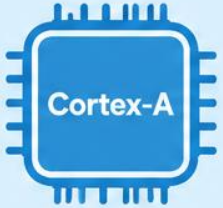
Family	Typical application
Cortex-M	Microcontrollers
Cortex-R	Real-time systems
Cortex-A	Application processors



Microcontrollers
(low power, real-time)



Real-time systems
(high reliability)

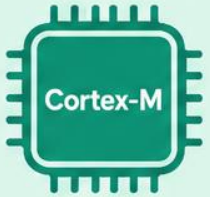


Application processors
(high performance)

Cortex-M

Typical:

- STM32
- NXP MCUs
- Nordic MCUs
- Microchip/other ARM MCUs



Used in small, low-power embedded systems.

Cortex-A

Typical:

- Smartphones
- Embedded Linux systems
- Raspberry Pi-class systems



Used in high-performance applications with rich OS (e.g., Linux, Android).

The same ARM architecture family is used for different needs, from small MCUs to powerful application processors.

Cortex-M vs Cortex-A

Cortex-M

Designed for:

- Low power
- Fast interrupts
- Deterministic response
- Microcontroller applications

Usually:

```
Cortex-M
    ↓
Bare-metal / RTOS
```

Cortex-A

Designed for:

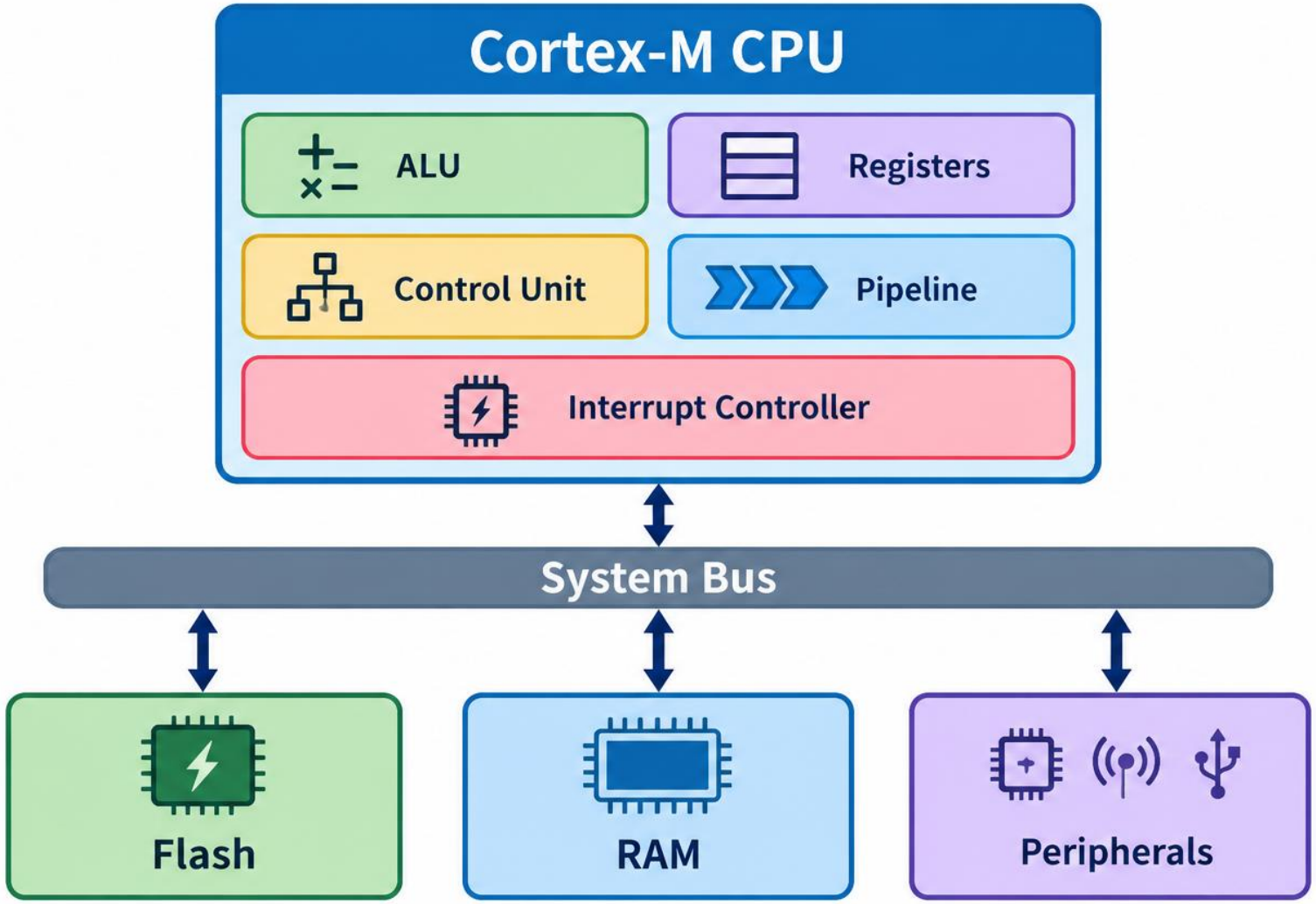
- Complex applications
- Virtual memory
- High-performance operating systems

Usually:

```
Cortex-A
    ↓
Linux / Android
```

Cortex-M Architecture

A simplified Cortex-M system



What Is the ALU?

ALU = Arithmetic Logic Unit

It performs operations such as:

- Addition
- Subtraction

- AND
- OR
- XOR

- Comparison
- Shift

Example:

```
result = a + b;
```

The addition is performed by the ALU.

5. REGISTERS

What Is a Register?

A register is a very small and very fast storage location inside the CPU.

Registers hold:

- Data
- Addresses
- Intermediate results
- Program information
- Processor status

When the CPU is performing an operation, it often works with values stored in registers.

Two different levels.

- CPU registers such as R0, R1, R2 and so on.
- Peripheral registers such as GPIO control registers.

Cortex-M Registers

Cortex-M processors have:

R0 R1 R2 ... R12

R13

R14

R15

Special registers:

- R13 → Stack Pointer (SP)
- R14 → Link Register (LR)
- R15 → Program Counter (PC)

General-Purpose Registers

Registers:

`R0 ... R12`

are mainly used to store:

- Variables
- Function arguments
- Intermediate values
- Addresses

Example:

```
int a = 10;
int b = 20;
int c = a + b;
```

The compiler may use CPU registers to hold these values.

6. PROGRAM COUNTER

Program Counter

The **Program Counter (PC)** contains the address of the instruction being executed or fetched.

Conceptually:

```
PC
 ↓
Instruction
 ↓
Execute
 ↓
PC changes
 ↓
Next instruction
```

Example:

```
0x08000100   instruction
0x08000104   instruction
0x08000108   instruction
0x0800010C   instruction
```

The PC moves through the program.

6. PROGRAM COUNTER

Why Is the PC Important?

Suppose the program executes:

```
printf("Hello");
```

The CPU needs to know:

Where is the next instruction?

The answer is provided by the **Program Counter**.

If the PC becomes invalid, the processor may execute unintended instructions and crash.

Stack Pointer

The stack is a special area of memory used during program execution.

The **Stack Pointer (SP)** points to the current stack location.

The stack is used for:

- Function calls
- Local variables
- Saved registers
- Interrupt context
- Temporary data

7. STACK POINTER

Function Call and Stack

Example:

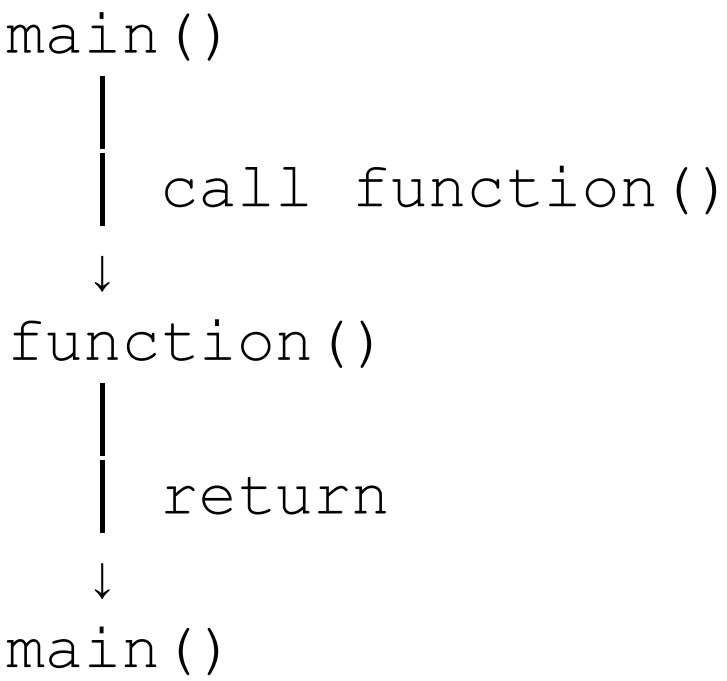
```
void function(void)
{
    int x = 10;
}
```

```
Before function
      ↓
Function call
      ↓
Stack changes
      ↓
Local variable
      ↓
Function returns
      ↓
Stack restored
```

8. LINK REGISTER

Link Register

The **Link Register (LR)** stores information needed to return from a function call.



The processor needs to remember where execution should continue.

9. STATUS REGISTER

Status Information

The processor maintains status information about execution.

Examples of flags include:

- Zero
- Negative
- Carry
- Overflow

These flags can be affected by arithmetic operations.

9. STATUS REGISTER

Example: Zero Flag

Suppose:

```
5 – 5 = 0
```

The result is zero.

The processor can set the **Zero flag**.

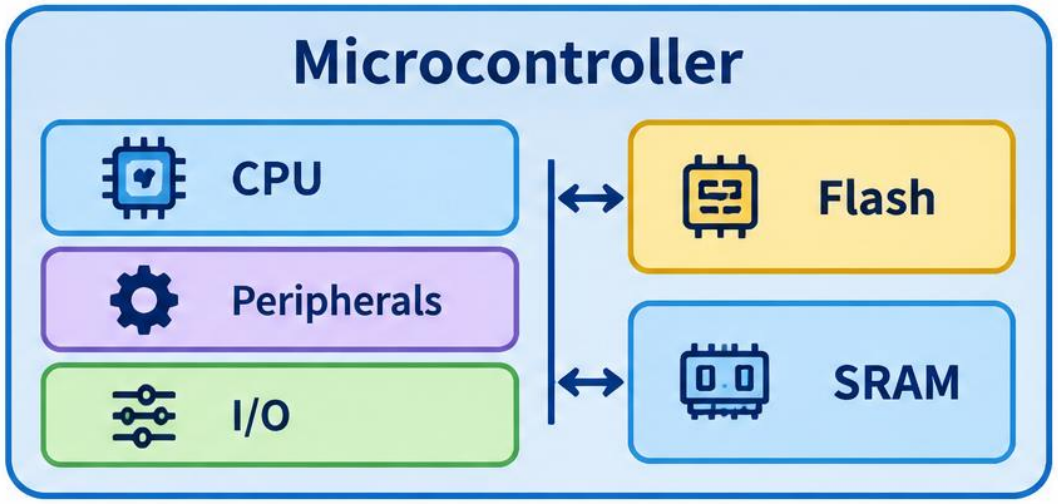
Then a conditional instruction can use this information.

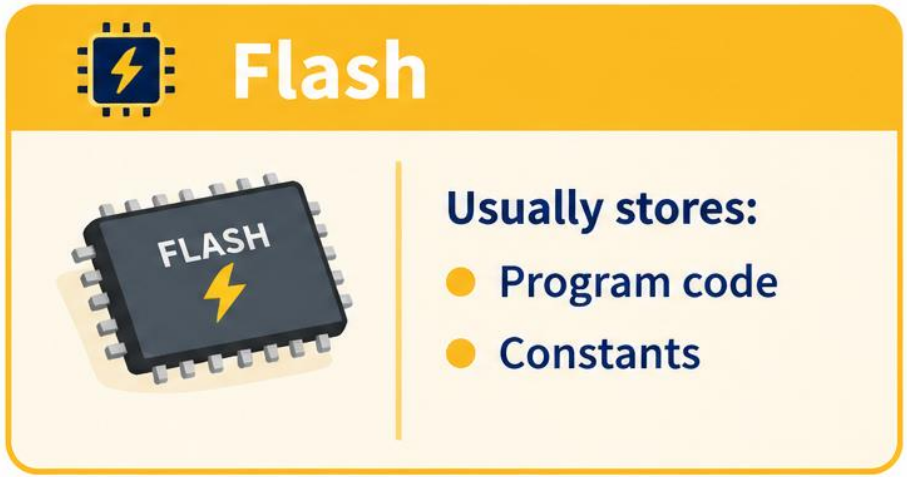
Conceptually:

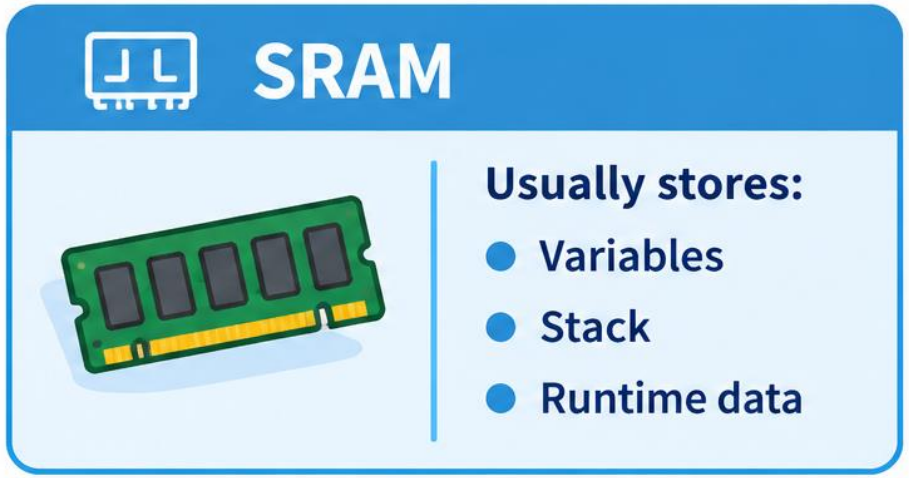
```
Compare
   ↓
Zero flag?
   ↓
Yes → branch
No  → continue
```

Embedded Memory

A microcontroller commonly contains:







Flash vs RAM

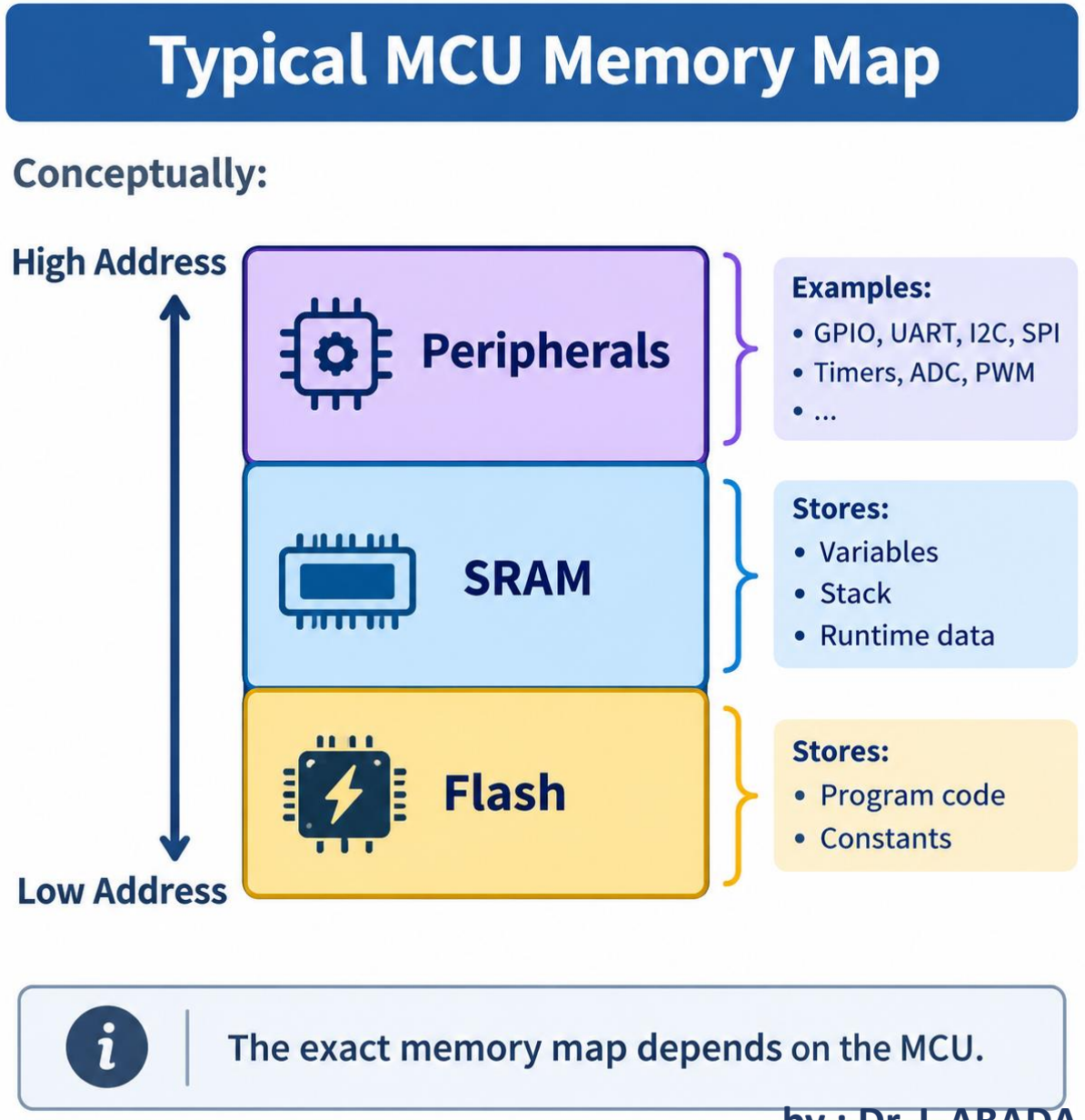
Flash	RAM
Non-volatile	Volatile
Stores program	Stores runtime data
Keeps data without power	Loses data without power
Usually slower to write	Fast read/write
Limited write endurance	No similar write-cycle concern

10. MEMORY

The processor uses addresses to access memory and peripherals.

- Some address ranges correspond to Flash.
- Other ranges correspond to SRAM.
- Other ranges are mapped to peripherals.

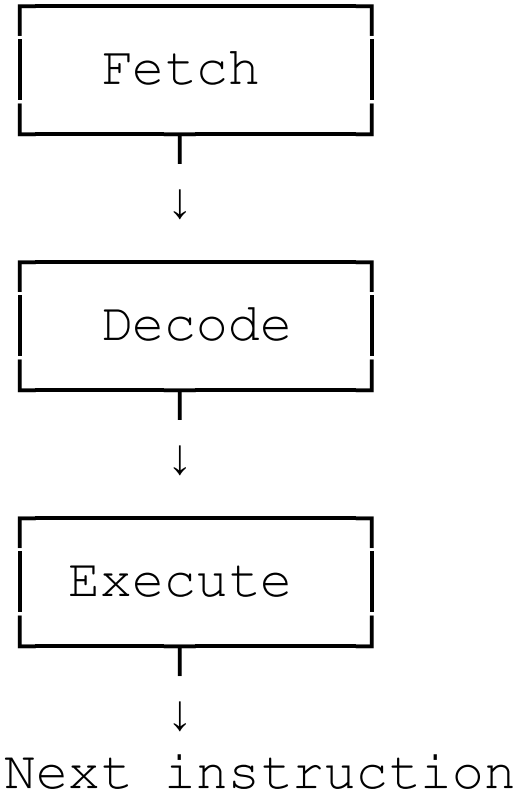
- **Peripherals can appear to the CPU as memory-mapped addresses.**
- **Software can control hardware by reading and writing specific addresses.**
- **One of the fundamental ideas behind register-level programming.**



11. INSTRUCTION EXECUTION

Instruction Cycle

A simplified processor cycle:



This process repeats continuously.

What Is Pipelining?

Pipelining allows different instructions to be processed at different stages simultaneously.

Without pipelining:

```
Instruction 1: Fetch → Decode → Execute
Instruction 2: Fetch → Decode → Execute
```

With pipelining:

```
Cycle        1        2        3        4
I1         Fetch    Decode   Execute
I2                  Fetch    Decode   Execute
I3                           Fetch    Decode
```

While one instruction is being executed, another instruction can be fetched. Another one can be decoded.

So different instructions are processed at different stages at the same time.

This improves the overall throughput of the processor.

Why Pipeline?

Without pipeline:

```
Instruction 1 → complete
Instruction 2 → complete
Instruction 3 → complete
```

With pipeline:

```
I1 → Fetch
I2 →       Fetch
I3 →             Fetch
```

Different instructions are processed simultaneously at different stages.

Important

Pipelining does **not** necessarily make one instruction execute instantaneously.

It improves the overall throughput.

13. INTERRUPT

Why Interrupts?

Imagine an MCU waiting for a button.

Method 1 — Polling

```
while (1)
{
    if (button_pressed())
    {
        handle_button();
    }
}
```

The CPU continuously checks the button.

Method 2 — Interrupt

The hardware informs the CPU when the event occurs.

```
Button event
      ↓
Interrupt
      ↓
CPU
      ↓
Interrupt handler
```

Polling vs Interrupt

Polling	Interrupt
CPU checks continuously	Hardware signals CPU
Simple	More complex
Can waste CPU time	Efficient for events
Response depends on polling	Fast response
Easy to understand	Important in real-time systems

14. INTERRUPT HANDLER

Interrupt Service Routine

An interrupt handler is a function executed when a specific interrupt occurs.

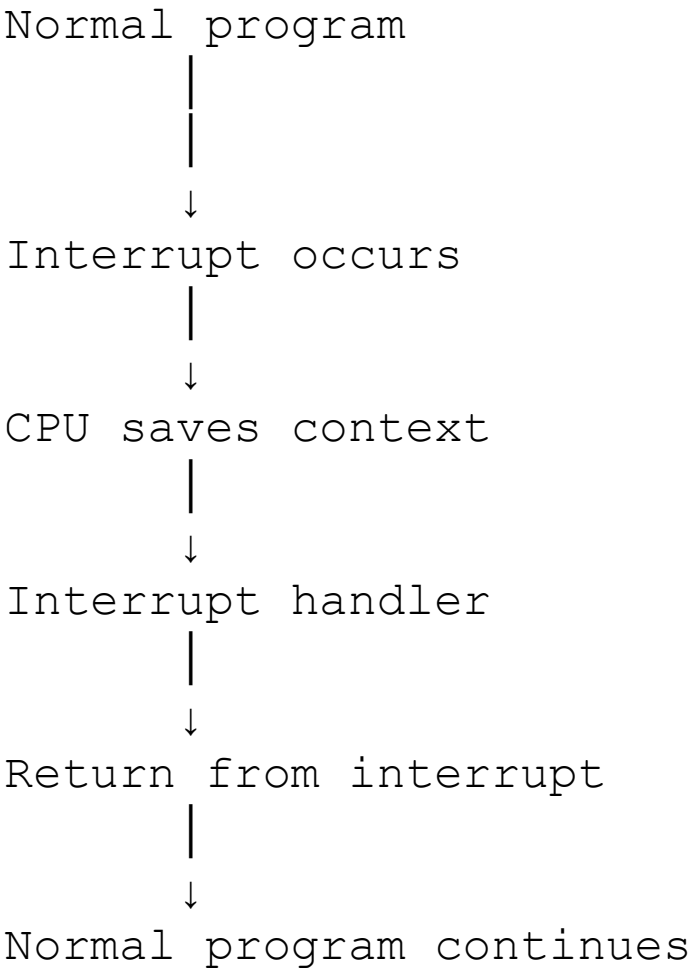
Conceptually:

```
void button_interrupt_handler(void)
{
    button_event = 1;
}
```

The exact syntax depends on the MCU/framework.

14. INTERRUPT HANDLER

Interrupt Flow

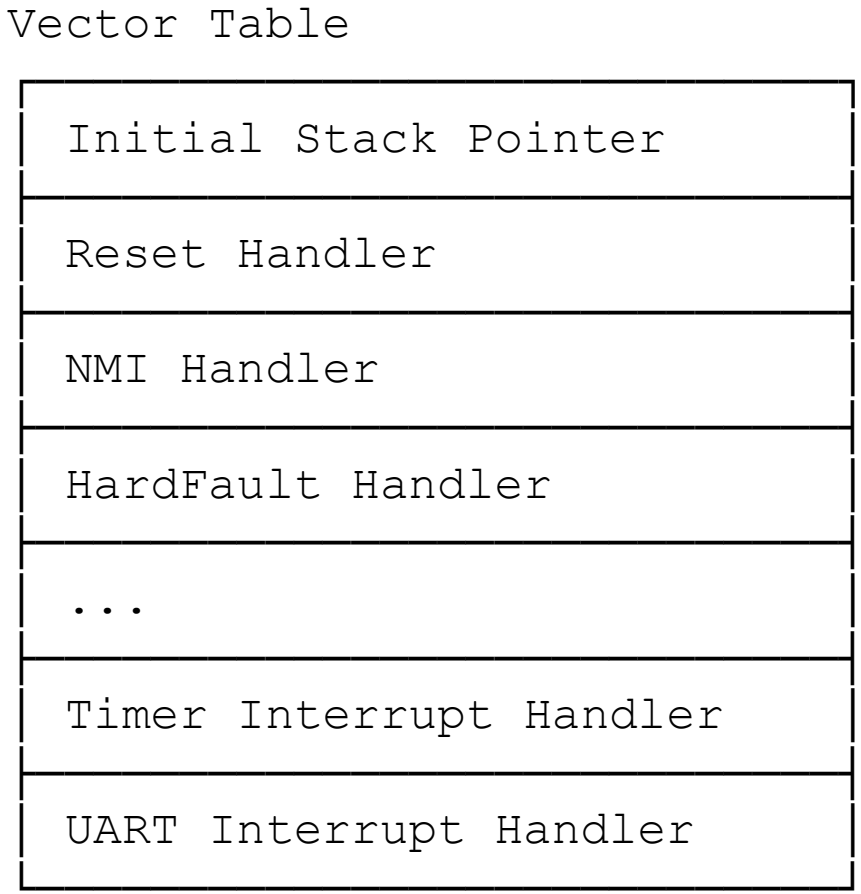


15. INTERRUPT VECTOR TABLE

What Is a Vector Table?

The **interrupt vector table** contains addresses associated with processor startup and exception/interrupt handlers.

Conceptually:



15. INTERRUPT VECTOR TABLE

Why the Vector Table?

Suppose UART generates an interrupt.

The processor needs to know:
Which function should I execute?
The vector table provides the corresponding handler address.

```
UART event
    ↓
Interrupt number
    ↓
Vector table
    ↓
UART handler
```

Interrupts and Exceptions

Cortex-M processors handle several types of exceptional events.

Examples:

Inter/Except	Meaning	Explanation
Reset	Restart the processor	The microcontroller starts or restarts its program.
NMI	Non-Maskable Interrupt	An interrupt that the processor normally cannot ignore or disable using the usual interrupt-disable mechanism.
HardFault	Critical processor fault	An exception that occurs when the processor encounters a serious error or cannot handle another fault.
Memory-related faults	Memory access errors	Exceptions caused by invalid memory access, such as trying to access a protected memory area. Available fault types depend on the processor core.
SysTick	System Timer	A timer that generates periodic interrupts, often used to keep track of time or schedule tasks.
External interrupts	Interrupts from external events	Interrupts triggered by events such as a button press or a signal on a GPIO pin.

Important distinction

An **external interrupt** is generally generated by a peripheral or external event.

An **exception** is a broader processor-level concept.

Interrupt Priority

Some systems need different priorities.

Example:

```
Motor control interrupt
        Priority: HIGH

UART reception
        Priority: MEDIUM

Button
        Priority: LOW
```

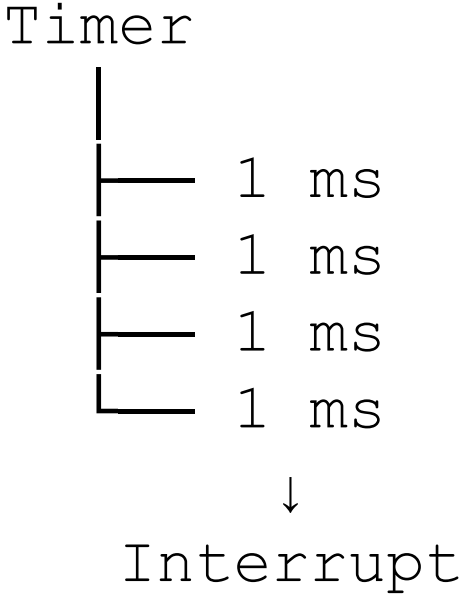
If a high-priority interrupt occurs while a lower-priority interrupt is executing, the processor may respond according to the configured priority scheme.

18. TIMER INTERRUPTS

Timer Example

A timer can generate periodic interrupts.

Example:



This can be used for:

- Periodic sensor acquisition
- Control loops
- Timekeeping
- Scheduling
- Software timers

Periodic Task Example

Suppose:

```
Sensor task = every 10 ms
```

The timer can trigger:

```
0 ms    → interrupt
10 ms   → interrupt
20 ms   → interrupt
30 ms   → interrupt
```

The interrupt can notify software that it is time to perform the task.

19. PRACTICAL EXAMPLE — STM32

STM32F411 Example

Consider an STM32F411-based board.

The MCU contains:

- ARM Cortex-M4 core
- Flash memory
- SRAM
- GPIO
- Timers
- ADC
- UART
- SPI
- I^2C
- Interrupt controller

This is a typical example of an ARM-based embedded system.

19. PRACTICAL EXAMPLE — STM32

STM32 Software Stack

A simplified architecture:

```
Application
     ↓
HAL / Drivers        (HAL : Hardware Abstraction Layer)
     ↓
CMSIS                (CMSIS = Cortex Microcontroller Software Interface Standard
                      It provides standardized interfaces for Cortex-M software.)
     ↓
Cortex-M4
     ↓
STM32 Hardware
```

we can work at different abstraction levels:

high-level libraries, drivers, or directly with registers.

Hardware Registers

Peripherals are controlled through registers.

For example, GPIO (port A) has several registers.

```
GPIOA->MODER (Mode Register)
GPIOA->ODR(Output Data Register)
GPIOA->IDR   (Input Data Register)
```

These registers configure and control GPIO behavior.

Why Learn Registers?

Using high-level libraries is useful.

However, understanding registers helps you understand:

- What the MCU actually does
- How peripherals work
- Why configurations matter
- Debugging
- Performance
- Datasheets and reference manuals

This is especially important for embedded engineers.

21. PRACTICAL EXAMPLE — GPIO

LED Control

Suppose an LED is connected to a GPIO.

```
Software
    ↓
GPIO register
    ↓
GPIO output
    ↓
LED
```

Code at a high level:

```
LED_ON();
```

At a lower level:

```
LED_ON()
    ↓
GPIO register modification
    ↓
Electrical signal
    ↓
LED turns ON
```

22. FROM C CODE TO HARDWARE

Compilation Process

When writing:

```
int main(void)
{
    LED_ON();

    while (1);
}
```

the MCU cannot directly execute the C source code.

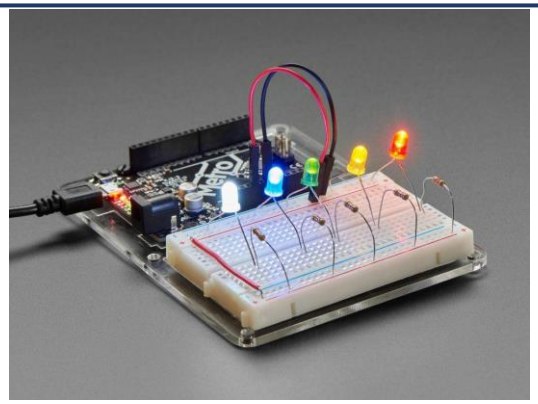
The development process is approximately:

```
C source
   ↓
Preprocessor
   ↓
Compiler
   ↓
Assembly / object code
   ↓
Linker
   ↓
Executable image
   ↓
HEX / BIN
   ↓
Flash MCU
   ↓
CPU executes machine instructions
```

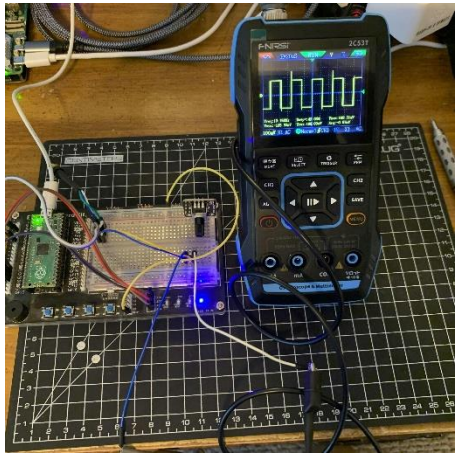
Embedded Debugging

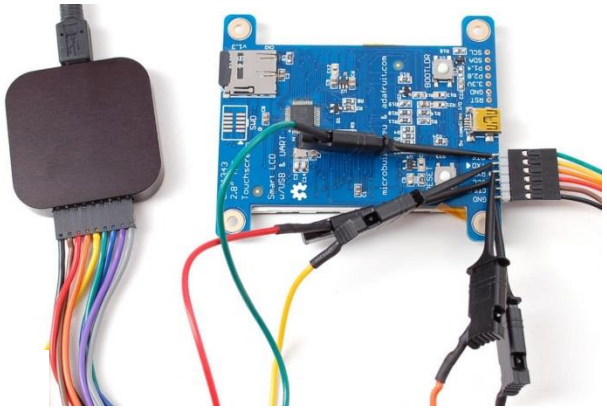
Debugging can be performed using:

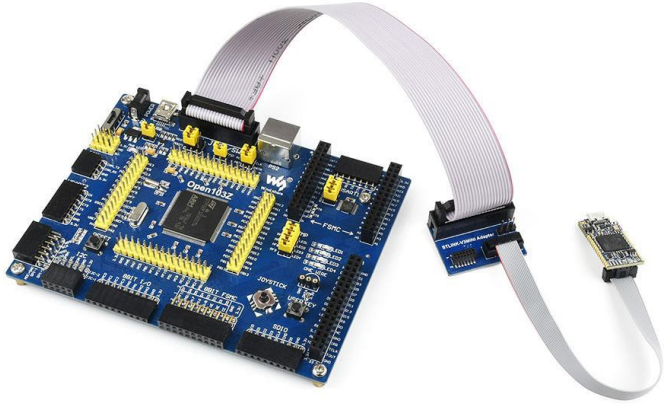
- Serial output (ex: FTDI)
- LEDs
- Logic analyzer
- Oscilloscope
- Debugger
- JTAG/SWD
- Breakpoints
- Watch variables
- Register inspection

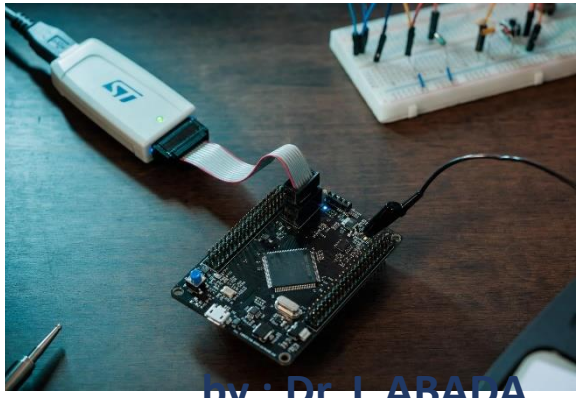












23. DEBUGGING : SWD and JTAG

JTAG

A standard interface historically used for:

- Debugging
- Programming
- Boundary scan

SWD : Serial Wire Debug

Common on Cortex-M devices.

Typical signals:

```
SWDIO
SWCLK
GND
RESET (optional/useful)
```

JTAG/SWD: These are communication interfaces used by the ST-LINK to communicate with the microcontroller.

Debugger: The ST-LINK allows the debugging software to control the processor, pause the program, and inspect variables and registers.

25. COURSE SUMMARY

Key Takeaways

Remember:

ARM

ARM provides processor architectures widely used in embedded systems.

Cortex-M

Designed for microcontroller applications.

Registers

Provide fast CPU storage and control information.

PC

Points to program execution.

SP

Manages the stack.

LR

Helps return from function calls.

Pipeline

Improves instruction throughput.

Interrupts

Allow the CPU to react to events efficiently.

Vector table

Associates interrupt/exception entries with handlers.